T.C.
SAKARYA ÜNİVERSİTESİ

BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

PROGRAMLAMA DİLLERİNİN PRENSİPLERİ ÖDEV RAPORU

ŞEHİR NÜFUS SİMÜLASYONU

B241210040 - Ahmet Faruk İKİZ

SAKARYA

Nisan, 2026

Programlama Dillerinin Prensipleri Dersi

PDP 1.Ödev Şehir Nüfus Simülasyonu

Ahmet Faruk İkiz[b241210040 1A*]

Özet

Ödevde İstenenler:

Ödevde bir string dizisi şeklinde alınan 2 basamaklı sayıların kendisi, birler ve onlar basamağı gibi özelliklerinden her biri şehirdeki nüfus (kendisi), ilçe (onlar) ve mahalle (birler) sayısını temsil etmektedir. Bu sayılar parçalanarak, şu kurallar: "Mahalle sayısı, her ilçede eşit olacak şekilde dağılabilir. Toplam nüfus, her mahallede eşit olacak şekilde dağılabilir." garanti edilecek şekilde tekrardan hesaplanarak simülasyonun başlangıcına uygun hale getirilmesi gerekmektedir. Hesaplanan sayılar sonucunda java-faker kütüphanesi yardımıyla rastgele değerli Sehir, Ilce, Mahalle, Kisi sınıfları oluşturulacaktır.

Ardından başlangıçta kullanıcıdan alınan tur sayısı kadar simülasyon yürütülecektir. Her tur için kişiler 1 yıl yaşlanacaktır ve şehrin nüfusu "mevcutNüfus = (birler+onlar) * mevcutNüfus" kuralına göre artacaktır. Her tur sonuna gelindiğinde şehrin nüfusu 4 basamaklı olduysa bölünecektir. Şehirde birden çok ilçe varsa; bu ilçeler tam bölünüyorsa eşit, bölünmüyorsa fazlalık eski şehirde kalacak şekilde eski şehirdeki ilçeler yeni şehire mahalleleri ve kişileriyle beraber aktarılmalıdır. Bölünme kuralları daha sonra detaylıca açıklanacaktır.

Ödevde Uyguladığım Yaklaşım:

Görülebileceği üzere aslında ödev birçok farklı aşamadan meydana gelmektedir. Bu aşamaları tek bir sınıfa yığmak kodun yazılışı ve anlaşılmasını epey zorlaştıracaktır. Ödevi geliştirirken SRP ve modülerlik açısından pek çok servis ve araç sınıfından yararlandım.Modellerin kendi iç hesaplamalarını dışa yansıtmayarak Oyun sınıfını daha temiz ve anlaşılabilir kıldım.

© 2026 Sakarya Üniversitesi.
Bu rapor benim özgün çalışmamdır. Faydalanmış olduğum kaynakları içeresinde belirttim. Herhangi bir kopya işleminde sorumluluk bana aittir.

Anahtar Kelimeler: Java, Clean Code, Concepts of Programming Languages, Object Oriented Programming

1. GELİŞTİRİLEN YAZILIM

1.1 Modülerlik

Ödevde araclar, main, modeller, motor, servisler olmak üzere 5 package kullandım:

- Araclar: İçerisinde durum tutmayan karışık hesaplamaları ve kodları servis sınıflarından soyutlayan statik metotlara sahip util sınıflarını barındırır.
- Main: Main sınıfı.
- Motor: Oyun sınıfını ve bu sınıfın kullanacağı interfaceleri barındırır.
- Modeller: İçerisinde veri tutan ve kendi iç verilerini yöneten model sınıflarını barındırır.
- Servisler: Oyun sınıfının görev yükünü hafifleterek mödülerlik sağlayan alt yönetim sınıfları.

1.2 Programın genel akışı ve Sınıflar

Main sınıfı:

Kullanıcıdan tur sayısı ve 2 basamaklı sayıları barındıran string alınır. Ardından BaslangicStringCevirici sınıfının statik stringiSayiyaCevir metoduyla bu string integer dizisine dönüştürülür. Sonrasında oyun sınıfı oluşturularak kurucu metoduna tur sayısı, sayılar dizisi ve uygun yazdırma servisi gönderilir. Oyun sınıfının baslat() metodu çağrılır. Aynı zamanda main sınıfında global hata kontrolü yapılır. Oyun sona erince enter tuşuna basılmasını bekler ardından simülasyon sonlanır.

1.2.1 Oyun sınıfı: Bu sınıfın alanları şunlardır: tur döngülerinin işletileceği sehirler listesi, oyunBaslaticiServis, IYazdirici tipinde yazdırma servislerini kapsayan interface referansı, sayiDizi, toplamTurSayisi. Main sınıfından aldığı sayı dizisini OyunBaslaticiServis'e gönderir, OyunBaslaticiServis bu sayıları alır ve içi dolu Sehir listesi döner. Oyun sınıfı bu sehirler listesini gerektiğinde yazdirici referansının metotlarıyla ekrana yazdırır. toplamTurSayisi kadar for döngüsüyle tur sayısının işletir. Yaşlanma ve nüfus artışı için Sehir sınıfının içerisinde bulunan nufusArttir() ve yaslandir() metotlarını kullanır. Eğer Sehir 4 basamaklı ise şehrin bolun() metodu yeni şehri Oyun sınıfına döndürür. Oyun sınıfı bu yeni şehri sehirler listesine ekler. Bu prensipe tell don't ask prensipi denir. Normal şartlarda Oyun sınıfı şehirden bilgi alır bu bilgilerle bir hesaplama yapar fakat bu prensipte bölünme mantığını doğrudan verilerin bulunduğu model sınıflarına aktarıyoruz. Tur sonu ve tur bitiminde yazdirici'nın turYazdir() metodunu sehirler argümanıyla çağırır. Turlar sona erince Oyun sınıfı içindeki oyunSonuSatirSutunSor() metodu çağrılır. Bu metod içinde oyuncudan satır ve sütun değeri (0'dan başlayacak şekilde) istenir, yazdirici'nın detayYazdir() metodu sehirler ve index argümanları gönderilerek çağrılır.

1.2.2 Model sınıfları: Yerleşkeden miras alırlar Bu sınıflar ve Kisi sınıfı kendi verilerini tutmalarının yanı sıra kendi verilerini yönetir ve işlerler. Her bir model sınıfı kendi alt sınıfının listesini barındırır (Yerleske ve Kisi sınıfı hariç). Bu hiyerarşi şu şekildedir: Sehir -> Ilce -> Mahalle -> Kisi.
nufusArttir(), nufusGuncelle(), bolun(), yaslandir(), bu metotlar bu hiyerarşiye dayanarak çalışır.

Sehir, Ilce ve Mahalle'de ad ve nufus alanları vardır bu bilgiler SahteVeriUretici sınıfının metotlarıyla oluşturulup super() ile Yerleske kurucusuna iletilir ve alanlarına atanır. Kisi sınıfı ad, soyad, yas, id alanlarına sahiptir. Burada program esnasında fazla kişi nesnesi üretileceğinden programın optimizasyonu için kişinin ad ve soyad bilgileri sadece ilk kez kullanıldığı zaman fakerdan çekilerek oluşturulmaktadır. Bu sebeple ad ve soyad atamaları kurucu fonksiyonda değil getter metotlarındadır.

1.2.3 OyunBaslaticiServis sınıfı: Görevi model sınıflarını oluşturup Oyun sınıfına sehirler listesi vermektir. Simülasyonda sayiDizi oluşturulduktan sonra turlar başlayana kadarki süreci tamamen bu servis yönetir. For döngüsü ile sayı dizisini dönerek sayıyı onlar ve birler basamağına ayırır. BaslangicKuralHesaplayici'nın metotlarını kullanarak sayi ve nufus bilgilerine ulaşır. Sayi ile ilçe ve mahalelleri nufus ile kişileri oluşturur. Her bir birimin oluşturulma süreci bu sınıf içerisinde de modülerlik için metotlara bölünmüştür.

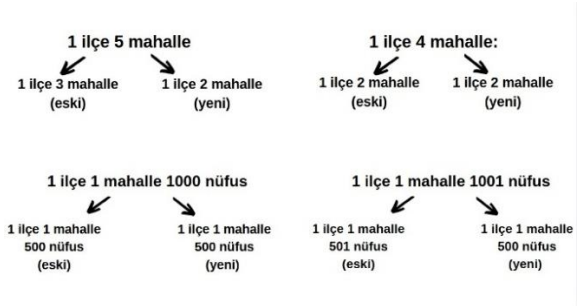
1.2.4 IYazdirici interface: SOLID ilkelerinden DIP'e dayanarak Oyun sınıfındaki yazdırıcı sınıfı interface referansıyla yazdım. Bu sayede test amaçlı oluşturduğum yazdırıcı sınıfıyla da çıktılar gözlemeyebildim.

1.3 Algoritmalar:

1.3.1 Bölünme algoritması:

Programda bir şehrin bölünmesinde şu 6 durum gözlenebilir: Çift Sayıda İlçe, Tek Sayıda İlçe, 1 ilçe çift sayıda mahalle, 1 ilçe tek sayıda mahalle, 1 ilçe 1 mahalle çift nüfus, 1 ilçe 1 mahalle tek nüfus.

Çift ve tek sayı durumları, tek sayı durumunda 1 eksik olarak aktarılacağından javada ilk hafta öğrendiğimiz tam sayı bölmesi kullanılarak rahatlıkla çözülebilir. aktarilacakSayi = BirimSayisi / 2; (Tek ise 1 eksik hesaplar küsüratı atar) 1 ilçe olduğu durumlarda ise ayrıştırma Şekil 1'deki gibi olacaktır.



Şekil 1. 1 İlçe Bölünme Örneği

Burada ilk akla gelen çözüm iç içe iki if else bloğu ile tüm koşulları tek tek kontrol etmektir. Fakat burada farkedileceği üzere aslında her durumda benzer bir mantık alt birime geçirilerek uygulanmaktadır.

bolun() metodunun çalışma mantığı:

1.Yeni birim oluştur.
2.Alt birim çok sayıdaysa alt birimi 2'ye böl yeni birime aktar. Tek sayıdaysa yeni alt birim oluştur bir alt birimin bolun() metodunu çağır dönen değeri yeni alt birime ata, yeni alt birimi yeni birime ekle.
3. Yeni birimin ve eski birimin nufusGuncelle() metodunu çağır
4. birimi return et.
Modellerin içinde bulunan bu mantığa sahip hiyerarşik bolun() metodları sayesinde Oyun sınıfı temiz ve okunabilir kalır. Program test edilebilir ve bakımı yapılabilir tutulur.

1.3.2 Nufüs artış Algoritması:

nufusArttir() **ve** nufusGuncelle() metodu: Sehir sınıfından başlayarak her model sınıfı bir alt sınıfın nufusArttir metodunu çağırıp artisOrani değişkenini aktarır. En son mahalle nüfusunu arttırır ve kendi nüfusunu return eder. Üst sınıflar alt sınıfların nüfuslarını toplayarak kendi nüfusunu hesaplar ve bir üste iletir. Aşağıdan yukarıya aktarımla şehrin nüfusu hem artmış hem de tekrar hesaplanmış olur nufusGuncelle() de benzer mantıkla çalışır.

2. ÇIKTILAR:

Girdiler: turSayisi: 2, sayilarString: 18 25 79 37 62 86 17 50

```
1.tur sonu:
[144]-[90]-[616]-[405]-[792]
[704]-[63]-[550]-[462]-[704]

2.tur sonu:
[576]-[810]-[2156]-[1350]-[4356]
[1408]-[567]-[1650]-[2464]-[1408]
[576]-[2156]-[675]-[4356]-[1408]
[1100]-[1232]-[1408]
```

Şekil 2. Girdi 1 içinYazdirServis çıktısı

```
1.tur sonu:
Şehir Adı        | Nüfus    | İlçe    | Mahalle
-----------------------------------------------------
Kansas           | 144      | 1       | 8
Georgia          | 90       | 2       | 6
Pennsylvania     | 616      | 4       | 4
Montana          | 405      | 3       | 9
Alaska           | 792      | 6       | 6
Tennessee        | 704      | 4       | 4
Vermont          | 63       | 1       | 7
Colorado         | 550      | 5       | 5
Massachusetts    | 462      | 3       | 3
Wisconsin        | 704      | 4       | 4
=====================================

2.tur sonu:
Şehir Adı        | Nüfus    | İlçe    | Mahalle
-----------------------------------------------------
Kansas           | 576      | 1       | 4
Georgia          | 810      | 2       | 6
Pennsylvania     | 2156     | 2       | 2
Montana          | 1350     | 2       | 6
Alaska           | 4356     | 3       | 3
Tennessee        | 1408     | 2       | 2
Vermont          | 567      | 1       | 7
Colorado         | 1650     | 3       | 3
Massachusetts    | 2464     | 2       | 2
Wisconsin        | 1408     | 2       | 2
Virginia         | 576      | 1       | 4
```

Şekil 3. Girdi 1 için TestYazdirServis çıktısı

Şekil 3'te görüldüğü üzere 1 ilçesi olan Kansas şehri 4 mahallesini yeni oluşan Virginia şehrine göndermiştir. Simülasyon sonunda satır ve sütun 0 3 girildiğinde 4 mahallesi olduğu görülebilir.

Girdiler: turSayisi: 3, sayilarString: 77 11 89 91 83 58 39 12 83 41 99

```
Başlangıç Durumu:
[77]-[11]-[88]-[99]-[88]
[55]-[45]-[12]-[88]-[44]
[99]

1.tur sonu:
[616]-[22]-[704]-[990]-[704]
[550]-[405]-[36]-[704]-[352]
[990]-[462]-[704]-[792]-[704]
[704]-[792]

2.tur sonu:
[2156]-[88]-[1408]-[5346]-[1408]
[1650]-[1350]-[324]-[1408]-[1232]
[5346]-[2464]-[1408]-[4356]-[1408]
[1408]-[4356]-[2156]-[1408]-[3564]
[1408]-[1100]-[675]-[1408]-[1232]
[3564]-[1232]-[1408]-[4356]-[1408]
[1408]-[4356]

3.tur sonu:
[11858]-[704]-[5632]-[35640]-[5632]
[5500]-[3375]-[972]-[5632]-[3080]
[35640]-[12320]-[5632]-[23958]-[5632]
[5632]-[23958]-[11858]-[5632]-[17820]
[5632]-[551]-[5400]-[5632]-[3080]
[17820]-[3080]-[5632]-[23958]-[5632]
[5632]-[23958]-[11858]-[704]-[5632]
[17820]-[5632]-[2750]-[3375]-[972]
[5632]-[3080]-[17820]-[12320]-[5632]
[23958]-[5632]-[5632]-[23958]-[11858]
[5632]-[17820]-[5632]-[551]-[2700]
[5632]-[3080]-[17820]-[3080]-[5632]
[23958]-[5632]-[5632]-[23958]
```

Şekil 4. Çıktı 2 için YazdirServisÇıktısı

11 -> 1+1 artış oranı = 2. 2*11 = 22 yeni nüfus. Şehirler hızla büyüyüp bölünmektedir ve gitdikçe az ilçe ve mahalleli yüksek nüfuslu şehirler görülmeye başlanacaktır.

3.SONUÇ

3.1 Öğrendiklerim

Bu ödevde SOLID prensiplerinden özellikle SRP, O/CP ve DIP üzerine pratik yapmış oldum. Ayrıca derste öğrendiğimiz tam sayı bölmesi, dizi indexi hesaplama, modüler döngüsel index arttırma, nesne referans gibi kavramları projede kullanarak pekiştirdim. Ayrıca java.util.list gibi hazır veri yapısı kütüphanelerini gerçek bir projede kullanmış oldum. Ödevi araştırırken simple factory design, tell don't ask, Dependency Inversion gibi kavramları öğrendim.

3.2 Projenin gerçek hayata katkıları

Bu mantıkta daha detaylı bir nüfus simulasyonuyla toplumun belirli bir yıl sonraki istatisksel bilgileri hesaplanarak geleceğe yönelik önlemler alınabilir.